AI-Based Web Crawler - Technical Documentation

Table of Contents

1. System Overview
2. Architecture
3. Core Components
4. Data Flow
5. AI Integration
6. Selector Management
7. Configuration
8. Error Handling
9. Output Structure
10. Performance Considerations

System Overview

Purpose

An intelligent web scraping system that uses AI models to automatically identify and extract web elements without hardcoded selectors. The system adapts to website structures dynamically through AI-driven selector categorization and selection.

Key Features

- AI-driven selector identification and categorization
- Dynamic element extraction without hardcoded selectors
- Multi-page review extraction with pagination
- Selector reusability across consistent page structures
- Comprehensive error handling and fallback mechanisms
- Batch processing of selectors to optimize AI queries

Target Platform

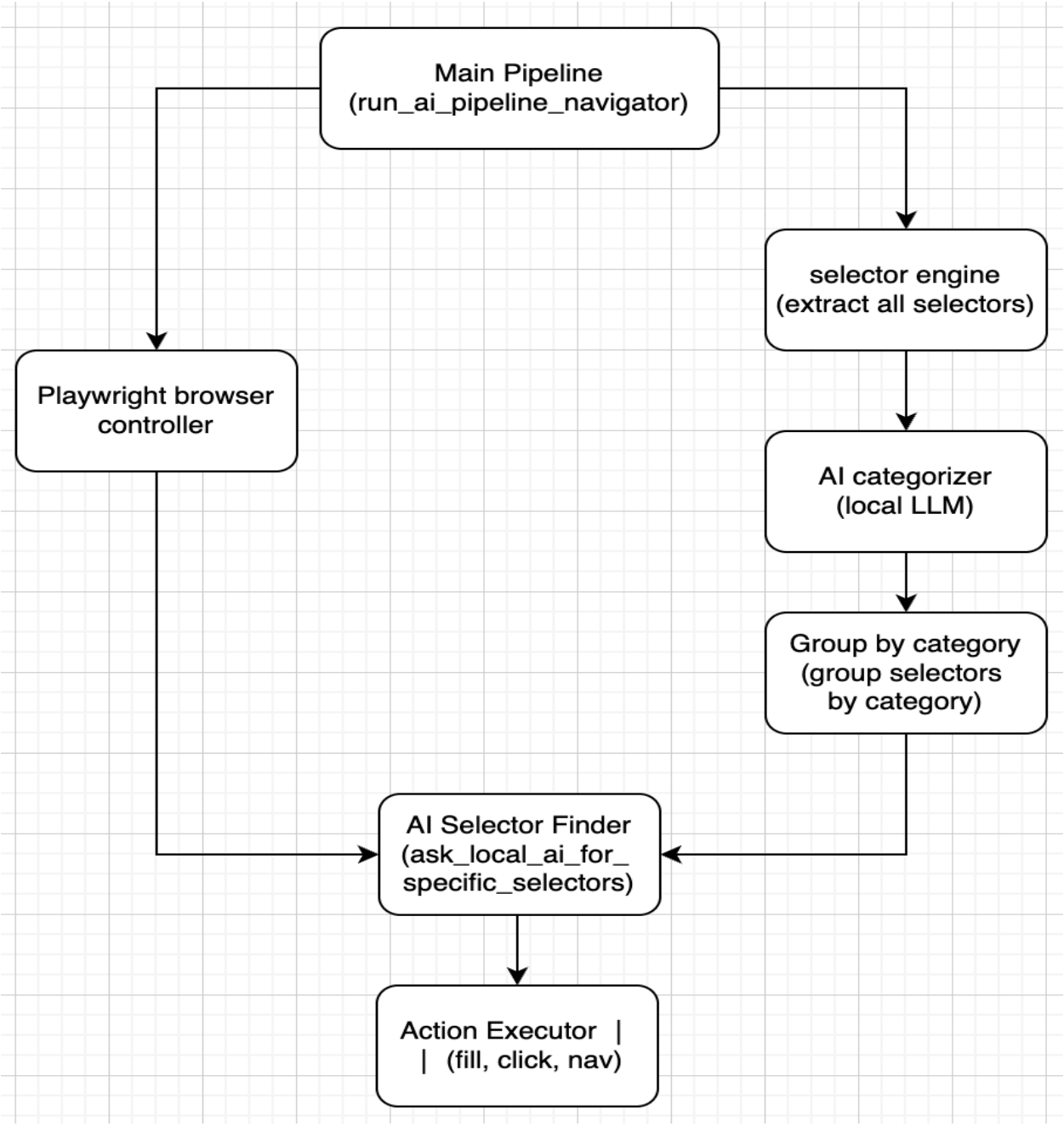
Amazon India or any other e- commerce platform (amazon.in) - Product search and review extraction

Technology Stack

- **Browser Automation**: Playwright (async)
- **HTML Parsing**: BeautifulSoup4
- **AI Models**:
    - Remote: Groq API (llama-3.1-8b-instant)
    - Local: LM Studio compatible models (qwen/qwen3-4b-2507, openai/gpt-oss-20b)
- **Language**: Python 3.8+
- **HTTP Client**: httpx (async)

Architecture

High-Level Architecture



Component Interaction Flow

Page Load → Extract Selectors → Categorize → Group → AI Query → Execute Action → Next Step
↑ ↓
└──┘
(Loop for each navigation step)

Core Components

1. extract_all_selectors()

Location: `test_extract_selector.py`

Purpose: Extracts all possible CSS selectors from HTML content

Functionality:

- Parses HTML using BeautifulSoup
- Identifies selectors by:
  - ID attributes
  - Class names
  - Name attributes
  - Type attributes
  - Custom data attributes
  - Input fields (with type, placeholder, name)
  - Buttons (with text content)
  - Links (with href)
  - Forms
- Assigns unique UUID to each selector for tracking
- Returns comprehensive selector data structure

Output Format:

```
{
  "id_selectors": [...],
```

```
  "class_selectors": [...],
  "input_selectors": [
    {
      "uuid": "uuid-123",
      "selector": "#email",
      "tag": "input",
      "type": "email",
      "name": "email",
      "placeholder": "Enter email"
    }
  ],
  "button_selectors": [...],
  ...
}
```

2. local_ai_selector_categorizer()

Location: `utilities_local_ai.py`

Purpose: Categorizes selectors into functional groups using AI

Categories:

- `navigation_layout`: Navigation menus, headers, footers
- `search_filters`: Search bars, filter controls
- `authentication_account`: Login, signup, account elements
- `category_listing`: Category navigation, breadcrumbs
- `product_details`: Product information, reviews, ratings
- `support_misc`: Help, support, miscellaneous elements
- `uncategorized_selector`: Unclassified elements

Process:

1. Receives all extracted selectors with UUIDs
2. Sends batch requests to local AI model
3. AI analyzes selector attributes and context
4. Returns categorized list with confidence scores

Output Format:

```
[
  {
    "uuid": "uuid-123",
```

```
    "category": "authentication_account",
    "confidence": 0.95
  }
]
```

3. group_selectors_by_category()

Location: `main.py`

Purpose: Merges extracted selector data with categorization results

Process:

1. Creates UUID lookup dictionary from extracted selectors
2. Iterates through categorized selectors
3. Enriches each selector with:
    - Full selector string
    - Tag information
    - Text content
    - Type-specific attributes (input_type, button_text, href)
    - Original extracted data
4. Groups by category with all metadata

Output Format:

```
{
  "authentication_account": [
    {
      "uuid": "uuid-123",
      "confidence": 0.95,
      "selector": "#ap_email",
      "tag": "input",
      "input_type": "email",
      "name": "email",
      "text_content": "",
      "selector_type": "input_selectors"
    }
  ]
}
```

4. ask_local_ai_for_specific_selectors()

Location: `main.py`

Purpose: Finds specific selectors for targeted actions using AI

Features:

- Batch processing (25 selectors per batch)
- Iterates until valid selector found
- Validates AI responses (rejects "none" or "null")
- Request types supported:
    - `sign_in`: Login/signin buttons
    - `username`: Email/username input fields
    - `password`: Password input fields
    - `submit_button`: Submit/continue buttons
    - `search_bar`: Search input fields
    - `reviews_link`: Links to review pages
    - `review_content`: Review text elements
    - `next_page`: Pagination next buttons

Process:

1. Divides selectors into batches of 25
2. Prepares clean selector list with relevant attributes
3. Sends targeted prompt to AI based on request type
4. Parses JSON response
5. Validates selector is not null/none
6. Returns on first valid match or exhausts all batches

Example Prompt Structure:

Find the BEST selector for the username/email input field.

SELECTORS:
[{selector: "#email", tag: "input", input_type: "email"}]

CRITERIA: Look for input_type "email"/"text", name containing "email"/"username". Prefer high confidence scores.

Return JSON: {"username_selector": "exact_selector_string"}

5. Review Extraction Pipeline

Location: `main.py` (Steps 28-33)

Process:

1. Extract review selectors from product page
2. Get AI recommendations for review content selectors
3. Extract reviews from product page (initial reviews)
4. Navigate to dedicated review page
5. Use same selectors to extract reviews (structure consistency)
6. Paginate through review pages using product page navigation selector
7. Deduplicate based on first 100 characters
8. Save to single JSON file

Key Optimization: Reuses product page selectors across all review pages since Amazon maintains consistent HTML structure.

Data Flow

Complete Pipeline Flow

1. HOMEPAGE
   ├─ Load amazon.in
   ├─ Extract all selectors
   ├─ Categorize selectors (AI)
   ├─ Group by category
   ├─ Find sign_in selector (AI)
   └─ Click sign in

2. LOGIN PAGE (Username)
   ├─ Extract all selectors
   ├─ Categorize selectors (AI)
   ├─ Group by category
   ├─ Find username selector (AI)
   ├─ Fill username
   ├─ Find continue button (AI)
   └─ Click continue

3. LOGIN PAGE (Password)
   ├─ Extract all selectors
   ├─ Categorize selectors (AI)
   ├─ Group by category
   ├─ Find password selector (AI)
   ├─ Fill password
   ├─ Find submit button (AI)
   └─ Submit login

4. MAIN PAGE
   ├─ Extract all selectors
   ├─ Categorize selectors (AI)
   ├─ Group by category
   ├─ Find search bar (AI)
   ├─ Enter search query
   └─ Submit search

5. SEARCH RESULTS
   ├─ Extract all selectors
   ├─ Categorize selectors (AI)
   ├─ Group by category
   ├─ Find first product (AI)
   └─ Click product

6. PRODUCT PAGE
   ├─ Extract all selectors
   ├─ Categorize selectors (AI)
   ├─ Group by category
   ├─ Find review content selectors (AI)
   ├─ Extract product page reviews
   ├─ Find reviews link (AI)
   ├─ Click reviews link
   └─ Store review & navigation selectors

7. REVIEW PAGES (Loop)
   ├─ Extract reviews using stored selectors
   ├─ Deduplicate reviews
   ├─ Find next page button using stored selector
   ├─ Click next (if available)
   └─ Repeat until no more pages

8. OUTPUT
   └─ Save all reviews to extracted_reviews/all_reviews.json

AI Integration

Local AI Setup

Prerequisites:

- LM Studio or compatible local AI server

- Running on `http://localhost:1234`
- OpenAI-compatible API endpoint

Supported Models:

- `qwen/qwen3-4b-2507`
- `openai/gpt-oss-20b`
- Any LM Studio compatible model

Configuration:

```
# Environment variables
LOCAL_AI_MODEL=qwen/qwen3-4b-2507

# Or pass directly
asyncio.run(run_ai_pipeline_navigator("qwen/qwen3-4b-2507"))
```

Remote AI (Groq)

Configuration:

```
# Environment variables
GROQ_API_KEY=your_api_key_here

# Used for legacy functions (find_username_selectors, etc.)
```

AI Query Optimization

Batch Processing:

- Processes 25 selectors per batch
- Reduces token usage
- Early termination on success
- Limits text content to 200 characters per selector

Response Validation:

```
# Validates response format
if isinstance(ai_response, dict) and len(ai_response) == 1:
    key, value = next(iter(ai_response.items()))
    if key and value and value.lower() not in ["none", "null"]:
        return ai_response  # Valid
```

Selector Management

Selector Storage Structure

```
extracted_data/
├── selectors/              # Raw extracted selectors
│   ├── home_all_selectors.json
│   ├── login_all_selectors.json
│   ├── password_all_selectors.json
│   ├── main_all_selectors.json
│   ├── search_results_all_selectors.json
│   └── product_page_all_selectors.json
├── categorized_selectors/   # AI categorized selectors
│   ├── home_categorized.json
│   ├── login_categorized.json
│   ├── password_categorized.json
│   ├── main_categorized.json
│   ├── search_results_categorized.json
│   └── product_page_categorized.json
└── grouped_selectors/       # Grouped by category
    ├── home_grouped.json
    ├── login_grouped.json
    ├── password_grouped.json
    ├── main_grouped.json
    ├── search_results_grouped.json
    └── product_page_grouped.json
```

Selector Reusability

Product Page Selectors: Reused across all review pages due to consistent HTML structure.

Benefits:

- No re-extraction on each page
- No re-categorization overhead
- Faster pagination
- Consistent data extraction

Implementation:

```
# Extract once from product page
review_selectors = product_page_grouped_selectors.get('product_details', [])
```

```
# Get AI recommendations once
ai_response = await ask_local_ai_for_specific_selectors(
    review_selectors, "review_content", model_name
)

# Reuse across all review pages
for page in range(1, max_pages):
    reviews = extract_using_cached_selectors()
    navigate_to_next_page()
```

Configuration

Environment Variables

```
# Required
AMAZON_USERNAME=your_email@example.com
AMAZON_PASSWORD=your_password
GROQ_API_KEY=your_groq_api_key

# Optional
LOCAL_AI_MODEL=qwen/qwen3-4b-2507  # Default model
```

Customizable Parameters

```
# In run_ai_pipeline_navigator()
url = "https://www.amazon.in"
search_product = "polo Tshirt"
max_pages = 10  # Review pagination limit

# In ask_local_ai_for_specific_selectors()
batch_size = 25  # Selectors per AI batch
text_content_limit = 200  # Character limit per selector

# Browser settings
browser = await p.chromium.launch(
    headless=False,  # Set True for production
    slow_mo=1000     # Delay between actions (ms)
)
page.set_default_timeout(15000)  # 15 seconds
```

Error Handling

Error Recovery Strategies

1. Selector Not Found:

```
if not selector:
    print("❌ Local AI could not find selector")
    # Try fallback categories
    # Or terminate gracefully
    await browser.close()
    return
```

2. AI Response Validation:

```
# Reject invalid responses
if value.lower() in ["none", "null"]:
    print("❌ Invalid selector value")
    continue  # Try next batch
```

3. Navigation Failures:

```
try:
    await page.click(selector)
    await page.wait_for_load_state("domcontentloaded", timeout=10000)
except Exception as e:
    print(f"❌ Failed: {e}")
    # Log error and continue or terminate
```

4. Pagination Edge Cases:

```
# Check if button is disabled
is_disabled = await next_button.get_attribute('disabled')
is_aria_disabled = await next_button.get_attribute('aria-disabled')
has_disabled_class = await next_button.evaluate(
    'el => el.classList.contains("a-disabled")'
)

if is_disabled or is_aria_disabled == 'true' or has_disabled_class:
    print("🏁 Reached last page")
    break
```

Logging Strategy

Verbose Output:

- Step-by-step progress indicators
- Selector counts per category
- AI response previews
- Success/failure status for each action
- Review extraction counts

Example Logs:

🔍 Starting AI search through 395 selectors in batches of 25
📊 Processing batch 1: selectors 1-25 of 395
🤖 Querying AI for batch 1...
✅ Found valid selector in batch 2: {'search_bar_selector': '#twotab...'}

Output Structure

Review Output Format

File: `extracted_reviews/all_reviews.json`


```
[
  {
    "text": "This polo t-shirt is excellent quality. The fabric is soft and comfortable. Highly recommended!",
    "selector_used": "[data-hook='review-body']",
    "source": "product_page",
    "url": "https://www.amazon.in/dp/B08XYZ123"
  },
  {
    "text": "Good fit but color fades after few washes. Not worth the price.",
    "selector_used": "[data-hook='review-body']",
    "source": "review_page_1",
    "url": "https://www.amazon.in/product-reviews/B08XYZ123"
  }
]
```


Fields:

- `text`: Review content (cleaned, deduplicated)
- `selector_used`: CSS selector that extracted this review
- `source`: Origin (product_page or review_page_N)
- `url`: Page URL where review was found

Intermediate Data Files

Grouped Selectors Example:


```
{
  "authentication_account": [
    {
      "uuid": "abc-123",
      "confidence": 0.95,
      "selector": "#ap_email",
      "tag": "input",
      "input_type": "email",
      "selector_type": "input_selectors"
    }
  ],
  "navigation_layout": [...],
  "product_details": [...]
}
```

Performance Considerations

Optimization Strategies

1. Selector Caching:

- Extract selectors once per page type
- Reuse across similar pages (review pagination)
- Reduces extraction overhead by ~80%

2. AI Query Batching:

- Process 25 selectors per request
- Early termination on success
- Reduces API calls significantly

3. Deduplication:

```
# Check first 100 chars to avoid duplicates
text_key = review_text.strip()[:100]
if not any(r['text'][:100] == text_key for r in all_reviews):
    all_reviews.append(review)
```

4. Asynchronous Operations:

```
# Use asyncio for browser and HTTP operations
await page.goto(url)
await page.click(selector)

# Offload blocking AI calls to thread pool
categorized = await loop.run_in_executor(
    None, local_ai_selector_categorizer, selectors, model_name
)
```

Resource Usage

Memory:

- Playwright browser instance: ~200-300MB
- Selector storage: ~1-5MB per page
- AI model (local): ~4-8GB VRAM

Network:

- Page loads: Variable (depends on content)
- AI API calls (remote): ~1-5KB per request
- Local AI: No network overhead

Time Estimates (per page):

- Selector extraction: 1-2 seconds
- AI categorization: 5-15 seconds (depends on selector count)
- AI specific selector query: 2-5 seconds per batch
- Page navigation: 2-5 seconds

Total Pipeline Time: ~5-10 minutes for complete flow with 10 review pages

---

Future Enhancements

Potential Improvements

1. **Multi-site Support**: Generalize categorization for different e-commerce platforms
2. **Parallel Processing**: Extract reviews from multiple products simultaneously
3. **Incremental Updates**: Store extraction state, resume from failure points
4. **Enhanced Deduplication**: Use similarity metrics beyond simple text matching
5. **Structured Review Data**: Extract ratings, dates, verified purchase status
6. **Selector Learning**: Build selector database from successful extractions
7. **Error Recovery**: Automatic retry with alternative selectors
8. **Performance Monitoring**: Track success rates per selector type

Scalability Considerations

Horizontal Scaling:

- Run multiple browser instances
- Distribute products across workers
- Centralized review aggregation

Vertical Scaling:

- Increase batch sizes for AI queries
- Cache frequently used selectors
- Optimize BeautifulSoup parsing

Troubleshooting

Common Issues

1. AI Returns Invalid Selectors:

- Check if selector format is correct (# for ID, . for class)
- Verify AI prompt is specific enough
- Increase text content limit in selector preparation

2. Pagination Fails:

- Verify next_page_selector is correctly identified
- Check for dynamic content loading
- Increase wait times after navigation

3. Reviews Not Extracted:

- Verify review_content_selectors are correct
- Check if page structure changed
- Try alternative category (uncategorized_selector)

4. Slow Performance:

- Reduce max_pages limit
- Use headless browser mode
- Optimize AI batch size
- Use faster local AI model

5. Login Failures:

- Verify credentials in .env file
- Check if Amazon requires CAPTCHA
- Increase timeouts for page loads
- Verify selector categories have authentication elements

Dependencies

playwright>=1.40.0
beautifulsoup4>=4.12.0
groq>=0.4.0
httpx>=0.25.0
python-dotenv>=1.0.0
lxml>=4.9.0

Install:

pip install -r requirements.txt
playwright install chromium

License & Usage

This is a technical documentation for an AI-based web crawler system designed for educational and research purposes. Ensure compliance with website terms of service and robots.txt when deploying in production environments.

Best Practices:

- Respect rate limits
- Implement proper delays between requests
- Handle robots.txt directives
- Use appropriate user agents
- Store credentials securely

Document Version: 1.0
Last Updated: September 2025
Maintained By: Development Team